

同济大学计算机系

词法分析、语法分析器设计实验报告

专业 计算机科学与技术

组 员 _____

组 员 _____

组 员 _____

授课老师 卫志华、高珍老师

目录

目录	2
一、实验背景与内容	3
1.1 实验背景	3
1.2 实验要求	3
1.3 Rust	4
1.3.1 Rust 词法要求	4
1.3.2 Rust 基础语法分析功能要求	4
1.3.3 Rust 扩展语法	5
二、项目分析设计过程	6
2.1 词法分析，语法分析过程	6
2.1.1 词法分析语法分析理论基础	6
2.1.2 词法分析理论与类 Rust 规则分析	6
2.1.3 语法分析理论：自上而下、LL(1) 与递归下降	7
2.2 项目数据结构	9
2.2.1 Token	9
2.2.2 AST 节点	10
2.2.3 符号表与作用域栈	10
2.2.4 错误分类 CompilerError / LexerError / ParserError / SemanticError	11
2.3 流程介绍	12
三、词法分析设计	12
3.1 词法分析器设计目标	12
3.2 词法分析模块输入输出	12
3.3 词法分析器主要实现逻辑	12
3.4 词法分析器关键函数实现	13
四、语法分析设计	13
4.1 语法分析器设计目标	13
4.2 语法分析器模块输入输出	14
4.3 语法分析器实现逻辑	14
4.4 递归下降分析结构及关键函数	15
4.5 表达式优先级处理及关键函数	16
五、语义分析设计	17
5.1 语义分析器设计目标	17
5.1.1 函数符号表检查	17
5.1.2 变量作用域管理	17
5.1.3 类型检查	17
5.1.4 可变性检查	17
5.1.5 控制流相关检查	17
5.2 语义分析器输入输出	17
5.3 语义分析器设计逻辑	17
5.4 语义分析器设计内容与关键函数	18

六、测试设计	18
6.1 测试设计设想	18
6.2 测试样例设计	18
6.2.1 应通过样例	18
6.2.2 应失败样例	18
6.2.3 词法边界测试	18
6.2.4 AST 结构测试	18
七、实验结果	19
7.1 词法分析结果	19
7.1.1 关键字和标识符边界	19
7.1.2 单字符符号和双字符符号	21
7.1.3 箭头和减号边界	22
7.2 语法分析结果	23
7.2.1 基础结构展示	23
7.2.2 表达式优先级	24
7.2.3 控制结构	27
7.2.4 错误	29
7.3 语义分析	29
7.3.1 语义正确案例	29
7.3.2 变量重复定义	30
7.3.3 参数匹配	30
7.3.4 赋值规则	31
7.3.5 函数缺少返回表达式	31
7.3.7 未定义错误	32
八、总结	33

一、实验背景与内容

1.1 实验背景

Rust 是一门能够安全高效地编写系统级程序的语言，在内存安全方面具有显著优势。自 2015 年以来，Rust 一直是开发者最爱的编程语言。

1.2 实验要求

写程序，实现对类 Rust 示例程序实现词法和语法分析，并输出分析结果。功能要求分为基础和扩展两部分。

1.3 Rust

1.3.1 Rust 词法要求

关键字: `i32` | `let` | `if` | `else` | `while` | `return` | `mut` | `fn` | `for` | `in` | `loop` | `break` | `continue`

标识符: (字母|_)(字母|数字|_)* (注: 不与关键字相同)

数值: 数字(数字)*

赋值号: `=`

算符: `+` | `-` | `*` | `/` | `==` | `>` | `>=` | `<` | `<=` | `!=` | `&` | `*`

界符: `()` | `{}` | `[]` | `[]`

分隔符: `;` | `:` | `,`

特殊符号: `->` | `.` | `..`

注释号: `/*` | `*/` | `//`

字母: `|a|...|z|A|...|Z|`

数字: `0|1|2|3|4|5|6|7|8|9|`

结束符: `#`

1.3.2 Rust 基础语法分析功能要求

变量, 类型和左值:

- `<变量属性> -> mut`
- `<类型> -> i32`
- `<左值> -> <可取引用>`
- `<可取引用> -> <可取元素>`
- `<可取元素> -> <ID>`

基础程序:

- `Program -> <声明串>`
- `<声明串> -> 空 | <声明> <声明串>`
- `<声明> -> <函数声明>`
- `<函数声明> -> <函数头声明> <语句块>`
- `<函数头声明> -> fn <ID> '(' <形参列表> ')'`
- `<形参列表> -> 空`
- `<语句块> -> '{' <语句串> '}'`
- `<语句串> -> 空`

语句:

- `<语句串> -> <语句> <语句串>`
- `<语句> -> ';'`

返回语句:

- `<语句> -> <返回语句>`
- `<返回语句> -> return ';'`

函数输入:

- `<形参列表> -> <形参> | <形参> ',' <形参列表>`
- `<形参> -> <变量属性> <ID> ':' <类型>`

函数输出:

- `<函数头声明> -> fn <ID> '(' <形参列表> ')' '->' <类型>`
- `<返回语句> -> return <表达式> ';'`

变量声明:

- <变量声明> -> <变量属性> <ID>
- <变量声明> -> <变量属性> <ID> ':' <类型>

变量声明语句:

- <语句> -> <变量声明语句>
- <变量声明语句> -> let <变量声明> ';' ;

赋值语句:

- <语句> -> <赋值语句>
- <赋值语句> -> <左值> '=' <表达式> ';' ;

基本表达式:

- <语句> -> <表达式> ';' ;
- <表达式> -> <加减表达式>
- <加减表达式> -> <项>
- <项> -> <因子>
- <因子> -> <NUM> | <左值> | '(' <表达式> ')'

增加比较运算:

- <表达式> -> <表达式> <比较运算符> <加减表达式>
- <比较运算符> -> '<' | '<=' | '>' | '>=' | '==' | '!='

增加加减运算:

- <加减表达式> -> <加减表达式> <加减运算符> <项>
- <加减运算符> -> '+' | '-'

增加乘除运算:

- <项> -> <项> <乘除运算符> <因子>
- <乘除运算符> -> '*' | '/'

增加函数调用:

- <因子> -> <ID> '(' <实参列表> ')'
- <实参列表> -> 空 | <表达式> | <表达式> ',' <实参列表>

选择结构:

- <语句> -> <if 语句>
- <if 语句> -> if <表达式> <语句块> <else 部分>
- <else 部分> -> 空

循环语句:

- <语句> -> <循环语句>

While 循环:

- <循环语句> -> <while 语句>
- <while 语句> -> while <表达式> <语句块>

1.3.3 Rust 扩展语法

变量声明赋值语句:

- <语句> -> <变量声明赋值语句>
- <变量声明赋值语句> -> let <变量声明> '=' <表达式> ';' ;

增加 else:

- <else 部分> -> else <语句块>

增加 **else if**:

- `<else 部分> -> else if <表达式> <语句块> <else 部分>`

for 循环:

- `<循环语句> -> <for 语句>`
- `<for 语句> -> for <变量声明> in <可迭代结构> <语句块>`
- `<可迭代结构> -> <表达式> '..' <表达式>`

loop 循环 (依赖 1.1, 拓展 5.0):

- `<循环语句> -> <loop 语句>`
- `<loop 语句> -> loop <语句块>`

增加 **break** 和 **continue**:

- `<语句> -> break ';' | continue ';'`

变量不可变属性:

- `<变量属性> -> 空`

不可变引用:

- `<类型> -> '&' <类型>`
- `<可取引用> -> '&' <可取引用>`

可变引用:

- `<类型> -> '&' mut <类型>`
- `<可取引用> -> '&' mut <可取引用>`

借用:

- `<左值> -> '*' <左值>`

函数表达式块

- `<函数表达式语句块> -> '{' <函数表达式语句串> '}'`
- `<函数表达式语句串> -> <表达式> | <语句> <函数表达式语句串>`

函数表达式块作为表达式

- `<可取元素> -> <函数表达式语句块>`

函数表达式块作为函数体

- `<函数声明> -> <函数头声明> <函数表达式语句块>`

二、项目分析设计过程

2.1 词法分析，语法分析过程

2.1.1 词法分析语法分析理论基础

本项目的目标是对类 Rust 示例程序进行词法分析和语法分析。类 Rust 程序虽然不是完整 Rust 编译器所支持的全部语言，但仍然保留了 Rust 中较典型的程序结构，例如函数定义、形参列表、变量声明、可变性标记 `mut`、类型标记 `i32`、返回语句、表达式、条件语句和循环语句。因此，本项目需要完成的不是简单的字符串匹配，而是按照编译前端的基本流程，先将源代码转化为 Token 序列，再根据文法规则判断这些 Token 能否构成合法程序。

2.1.2 词法分析理论与类 Rust 规则分析

词法分析的基本任务是将源程序中的字符序列划分为一个个具有类别的 Token。本项目

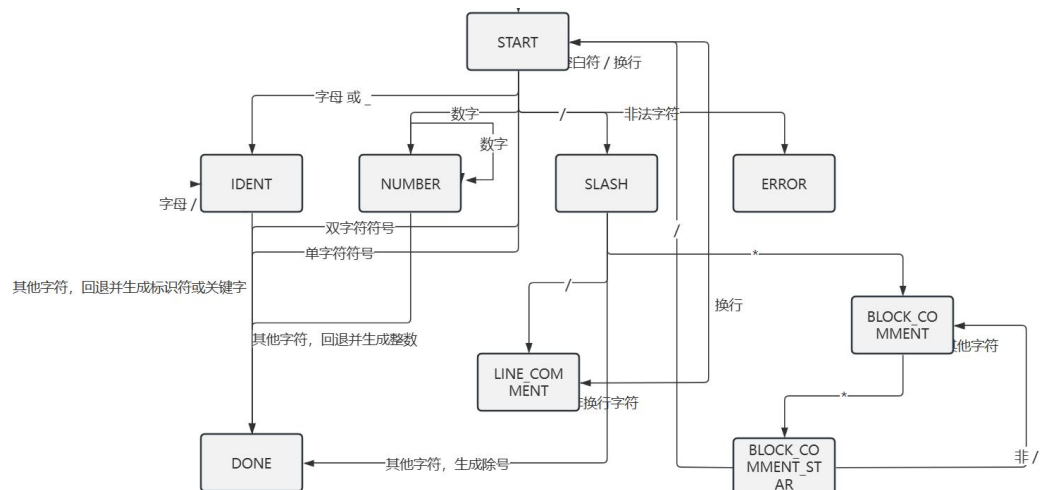
中的类 Rust 词法规则包括关键字、标识符、整数、运算符、界符、分隔符、特殊符号和注释等。这些规则大多具有正则语言特征，因此可以用有限状态自动机的思想进行识别。

例如，标识符规则为：

$(\text{字母} \mid _)(\text{字母} \mid \text{数字} \mid _)^*$

这意味着当扫描器遇到字母或下划线时，不能只读取当前一个字符，而应该继续向后读取所有连续的字母、数字和下划线，直到遇到不属于标识符组成部分的字符为止。读取完整词素之后，再判断它是否属于关键字表。这样设计的原因是，if 是关键字，但 if123 应该被整体识别为标识符，而不能拆分成 if 和 123。

词法分析状态转换图：



2.1.3 语法分析理论：自上而下、LL(1) 与递归下降

对语法分析器的定义是：对于给定输入串，判断它是否属于文法所定义的语言；也就是说，语法分析器要按照产生式规则识别输入串，并建立相应的语法结构。

类 Rust 的基础语法结构具有较明显的层次关系：

```

Program
  → 函数声明
    → 函数头 + 语句块
      → 多条语句
        → return / let / 赋值 / if / while / 表达式
    
```

语法分析方法包括自顶向下分析和自底向上分析，LR(1) 属于自底向上的分析方法，适合处理较复杂文法，考虑到本实验给出的类 Rust 文法结构较清晰，且本项目目标是完成词法分析、语法分析和分析结果输出，因此最终选择手写递归下降分析器。

左递归和回溯：

自上而下分析主要面临两个问题：左递归 和 回溯。

左递归指的是某个非终结符可以推导出以自身开头的符号串，例如：

$E \rightarrow E + T \mid T$

类 Rust 表达式文法中也存在类似问题。原始规则可能写成：

```

<表达式> -> <表达式> <比较运算符> <加减表达式>
<加减表达式> -> <加减表达式> <加减运算符> <项>
<项> -> <项> <乘除运算符> <因子>
    
```

这些都是左递归形式。如果直接写成递归下降程序，就会出现 `_parse_expression()` 还没

有消费任何 Token 就再次调用 `_parse_expression()` 的问题。因此，在实现中需要把它改写成适合递归下降的形式（{ ... } 表示可以重复 0 次或多次）：

表达式	-> 加减表达式 { 比较运算符 加减表达式 }
加减表达式	-> 项 { 加减运算符 项 }
项	-> 因子 { 乘除运算符 因子 }
因子	-> NUM ID '(' 表达式 ')' 函数调用

对应后续程序：

```
def parse_additive():
    left = parse_term()
    while current_token_is("+") or current_token_is("-"):
        op = advance()
        right = parse_term()
        left = BinaryExpr(left, op, right)
    return left
```

再考虑回溯：

回溯出现的原因是，当某个非终结符有多个候选式时，分析器无法根据当前输入符号唯一确定应该选择哪一个产生式，只能先试一个，失败后再退回去试另一个。课上给出的解决方向 LL(1) 分析，即从左到右扫描输入串，构造最左推导，并且每一步只向前看一个输入符号，由此来构造不带回溯的产生式。

LL(1)三个涵义：1、第一个 L：从左到右扫描输入串 2、第二个 L：构造最左推导 3、1：每次只向前看一个输入符号

LL(1) 分析的目标是让分析器在面对某个非终结符时，只根据当前 Token 就能确定应该使用哪条产生式，从而避免回溯。为了做到这一点，文法需要满足几个条件：首先不能含有左递归；其次，同一非终结符的不同候选式的 FIRST 集应当互不相交；如果某个候选式可以推出空串，还需要结合 FOLLOW 集判断是否可以选空产生式。课件中也给出了 LL(1) 文法条件：消除左递归，候选式 FIRST 集两两不相交，若存在 ϵ 候选，还需要满足 FIRST 和 FOLLOW 的相应条件。

在本项目中，Parser 不需要把每种语句都试一遍，而是根据当前 Token 直接分派到对应的解析函数。不过，也有一些地方需要额外处理。例如，当当前 Token 是标识符 ID 时，它可能是普通表达式语句，也可能是赋值语句，还可能是函数调用表达式：

```
a = 1;
a + 1;
foo();
```

这时仅看当前一个 Token ID 不一定够，需要适当查看下一个 Token。如果下一个 Token 是 =，可以判断为赋值语句；如果下一个 Token 是 (，可以在表达式解析中判断为函数调用；否则按普通表达式处理。严格说，这部分已经带有少量 lookahead 的工程处理，但整体仍然遵循 LL(1) 和递归下降的设计思想。

递归下降分析：

从类 Rust 语言结构看，程序、函数声明、语句块、语句和表达式之间具有明显的包含关系。程序可以展开为声明串，声明可以展开为函数声明，函数声明可以展开为函数头和语句块，语句块继续展开为多条语句。这种从整体到局部的结构，适合用递归下降方法进行描述。递归下降分析可以从 Program 开始，逐层下降到函数、语句和表达式，最终完成对整

个输入程序的识别。

因此，本项目在理论上采用“自上而下分析 +消除左递归+ LL(1) 文法处理回溯思想 + 递归下降实现形式”的路线。也就是说，递归下降是这种分析思想的程序化实现，用于说明这些理论如何转化为一组自顶向下的分析过程。

2.2 项目数据结构

2.2.1 Token

Token 是词法分析阶段的输出，也是语法分析阶段的输入。每个 Token 记录一个词法单元，主要包括 Token 类型、原始文本、行号和列号。Token 类型由 TokenKind 枚举表示，例如关键字、标识符、整数、运算符、界符和文件结束符等。行号和列号用于在出错时定位源代码位置。

对于程序语句 let mut a = 1;

LET	let
MUT	mut
IDENT	a
EQ	=
NUMBER	1
SEMICOLON	;

其中的 LET、MUT、IDENT、EQ、NUMBER 就是 TokenKind.

class TokenKind(Enum):	
IDENT = auto()	# 标识符，如变量名、函数名
NUMBER = auto()	# 整数字面量
LET = auto()	# 关键字 let
IF = auto()	# 关键字 if
FN = auto()	# 关键字 fn
PLUS = auto()	# 运算符 +
EQ = auto()	# 赋值符号 =
LPAREN = auto()	# 左括号 (
RPAREN = auto()	# 右括号)
EOF = auto()	# 输入结束标记

对于一个每一个 Token，除了类型，还保存原始文本和位置，这个记录单元是 Token:

@dataclass(frozen=True)	
class Token:	
kind: TokenKind	# Token 类型
lexeme: str	# 源码中的原始文本
line: int	# 所在行号

column: int	# 所在列号
-------------	--------

2.2.2 AST 节点

AST 是语法分析成功后生成的程序结构，是语法分析阶段的输出，也是语义分析阶段的输入。相比 Token 序列，AST 不再只是线性排列的单词，而是用树形结构表示程序的层次关系。例如函数定义中包含参数列表和函数体，函数体中包含语句，表达式又可以继续分解为左右子表达式。

以下为 AST 最终输出所需结构。

class FunctionDecl:	# 表示函数结构
name: str	# 函数名
params: list["Param"]	# 形参列表
return_type: Optional["TypeNode"]	# 返回类型，可为空
body: "Block"	# 函数体语句块

@dataclass	# 表示函数体里的一条变量声明语句
class LetStmt(Statement):	
is_mut: bool	# 是否带 mut
name: str	# 变量名
type_name: Optional[TypeNode]	# 显式类型，可为空
initializer: Optional["Expr"] = None	# 初始化表达式

@dataclass	
class BinaryExpr(Expr):	# 表示变量初始化时使用的表达式
left: Expr	# 左操作数
op: str	# 运算符
right: Expr	# 右操作数

2.2.3 符号表与作用域栈

符号表和作用域栈用于语义分析阶段保存上下文信息。函数符号表记录函数名、参数类型和返回类型，变量符号记录变量名、变量类型和是否可变。作用域栈 scopes 用来表示嵌套代码块中的变量可见范围，进入新代码块时压入新的作用域，离开代码块时弹出作用域。

例如 fn add(mut a:i32, mut b:i32) -> i32

FunctionSymbol 保存：

name: add
params: [i32, i32]
return_type: i32

例如 let mut a:i32;

VariableSymbol 保存：

name: a
type_name: i32

```
is_mut: true
```

```
@dataclass
class FunctionSymbol:
    name: str          # 函数名
    params: list[TypeNode]  # 参数类型列表
    return_type: Optional[TypeNode]  # 返回类型
```

```
@dataclass
class VariableSymbol:
    name: str          # 变量名
    type_name: Optional[TypeNode]  # 变量类型
    is_mut: bool       # 是否可变
```

scopes 是变量符号表的栈。

```
class SemanticAnalyzer:
    def __init__(self):
        # 全局函数符号表：函数名 -> 函数信息
        self.functions: dict[str, FunctionSymbol] = {}

        # 作用域栈：每一层保存当前代码块中的变量
        self.scopes: list[dict[str, VariableSymbol]] = []
```

2.2.4 错误分类 **CompilerError / LexerError / ParserError / SemanticError**

错误信息结构用于区分不同阶段产生的错误。这样主流程可以统一捕获编译前端相关错误，同时又能根据错误类型判断问题发生在哪个分析阶段。

```
class CompilerError(Exception):
    # 编译前端错误的统一基类
    pass

class LexerError(CompilerError):
    # 词法分析阶段错误
    pass

class ParserError(CompilerError):
    # 语法分析阶段错误
    pass

class SemanticError(CompilerError):
    # 语义分析阶段错误
    pass
```

2.3 流程介绍

源程序由以下结构组成：

```
输入程序
-> 词法分析 Lexer
-> Token 序列
-> 语法分析 Parser
-> AST 抽象语法树
-> 语义分析 SemanticAnalyzer
-> 通过 / 报错
```

三、词法分析设计

词法分析模块对应 `src/lexer.py` 和 `src/tokens.py`。

3.1 词法分析器设计目标

词法分析不判断完整程序结构是否正确，只负责把源码切分成带类型的 `Token`，为后续语法分析提供输入。

3.2 词法分析模块输入输出

词法分析器逐字符扫描源代码，将字符流转换为 `Token` 序列。每个 `Token` 包含四类信息：

```
kind    # Token 类型
lexeme  # 原始文本
line    # 行号
column  # 列号
```

3.3 词法分析器主要实现逻辑

- 跳过空白字符和注释。
- 字母或 `_` 开头时读取标识符，再判断是否为关键字。
- 数字开头时读取整数。
- 优先匹配双字符符号，如 `->`、`==`、`!=`、`>=`、`<=`、`...`。
- 再匹配单字符符号，如 `+`、`-`、`*`、`/`、`(`、`)`、`{`、`}`、`;`、`..`。
- 遇到无法识别的字符时抛出词法错误。

3.4 词法分析器关键函数实现

<code>tokenize()</code>	主控函数，从头到尾扫描源码，决定当前字符应该进入哪一种识别逻辑。
<code>_skip_whitespace_and_comments()</code>	跳过空格、换行、行注释和块注释。这样 Parser 后面不用关心注释。
<code>_read_identifier()</code>	连续读取字母、数字、下划线，然后再查 KEYWORDS 表，判断是关键字还是标识符。
<code>_read_number()</code>	连续读取数字，形成 NUMBER Token。

Lexer 的核心函数是 `tokenize()`。该函数维护当前扫描位置 `index`、当前行号 `line` 和列号 `column`，从源代码开头开始逐字符扫描。每轮循环先调用 `_skip_whitespace_and_comments()` 跳过空白字符和注释，因为这些内容不参与语法结构判断；随后根据当前字符类型选择不同的识别规则。

当当前字符是字母或下划线时，调用 `_read_identifier()` 连续读取字母、数字和下划线，得到完整词素后再查询 KEYWORDS 表。如果词素在关键字表中，则生成关键字 Token；否则生成 IDENT Token。这样可以保证 `if` 被识别为关键字，而 `if123` 被整体识别为标识符。

当前字符是数字时，调用 `_read_number()` 连续读取数字，生成 NUMBER Token。对于运算符和界符，程序先匹配双字符符号，如 `->`、`==`、`!=`、`>=`、`<=`、`..`，再匹配单字符符号。这种顺序是必要的，否则 `..` 会被错误识别成两个 `.`，`>=` 也可能被拆成 `>` 和 `=`。

四、语法分析设计

项目采用的是递归下降分析法。递归下降分析是一种自顶向下的语法分析方法，通常为文法中的非终结符设计对应的解析函数，通过函数调用关系体现文法结构。

4.1 语法分析器设计目标

按照预设文法消费 Token，若 Token 排列符合规则，则构造 AST；如果不符合，则报告语法错误。

例：

`let mut a:i32 = 1;`

得到：

LetStmt

`is_mut`: true

`name`: a

`type_name`: I32Type

`initializer`: NumberExpr(1)

4.2 语法分析器模块输入输出

接受 token，输出 AST。AST 节点定义在 `src/ast_nodes.py`。

4.3 语法分析器实现逻辑

- `parse_program()` 解析整个程序。
- `_parse_function_decl()` 解析函数定义。
- `_parse_param_list()`、`_parse_param()` 解析形参列表。
- `_parse_type()` 解析类型。
- `_parse_block()` 解析语句块。
- `_parse_statement()` 根据当前 Token 分派到不同语句解析函数。
- `_parse_expression()` 解析表达式。
- `_parse_comparison()`、`_parse_additive()`、`_parse_term()`、`_parse_factor()` 分层处理表达式优先级

可以用一个简单例子说明递归下降的分析过程：

```
fn test(mut a:i32) -> i32 {  
    return a + 1;  
}
```

对于这段程序，Parser 的分析过程如下：

```
parse_program()  
    发现当前 Token 是 fn  
    调用 _parse_function_decl()  
  
_parse_function_decl()  
    匹配 fn  
    匹配函数名 test  
    匹配 (  
    调用 _parse_param_list()  
    匹配 )  
    发现 ->  
    调用 _parse_type()  
    调用 _parse_block()  
  
_parse_block()  
    匹配 {  
    当前 Token 是 return  
    调用 _parse_statement()  
  
_parse_statement()  
    当前 Token 是 return  
    调用 _parse_return_stmt()
```

```

_parse_return_stmt()
    匹配 return
    调用 _parse_expression()
    匹配 ;

_parse_expression()
    调用 _parse_comparison()
        调用 _parse_additive()
            调用 _parse_term()
                调用 _parse_factor() 识别 a
            发现 +
            再调用 _parse_term()
                调用 _parse_factor() 识别 1
        形成 BinaryExpr(a + 1)

_parse_block()
    匹配 }

```

4.4 递归下降分析结构及关键函数

函数可以分为三组：

程序结构解读：

`parse_program()`

`_parse_function_decl()`

语句解析：

`_parse_statement()`

Parser 采用手写递归下降分析。每一类语法结构对应一个解析函数，函数之间的调用关系与文法层次基本一致。`parse_program()` 是语法分析入口，负责不断解析函数声明；`_parse_function_decl()` 负责匹配 `fn`、函数名、参数列表、返回类型和函数体；`_parse_block()` 负责解析由大括号包围的语句块；`_parse_statement()` 根据当前 `Token` 类型分派到 `let`、`return`、`if`、`while`、`for`、`loop`、`break`、`continue` 或表达式/赋值语句的解析函数。

这种设计没有构造 LR(1) 分析表，也没有 ACTION/GOTO 表，而是通过代码中的 `_match()`、`_expect()`、`_check()` 等函数直接消费 `Token`。优点是结构直观，便于和实验文法对应；缺点是需要手工处理左递归、优先级和部分 lookahead 情况。

例：

```

fn test(mut a:i32) -> i32 {
    let mut b:i32 = a + 1 * 2;
    return b;
}

```

`_parse_function_decl()` 识别函数 `test`。

`_parse_param_list()` 识别参数 `mut a:i32`。
`_parse_type()` 识别返回类型 `i32`。
`_parse_block()` 进入函数体。
`_parse_statement()` 识别 `let` 语句。

4.5 表达式优先级处理及关键函数

`_parse_expression()`
`_parse_comparison()`
`_parse_additive()`
`_parse_term()`
`_parse_factor()`

表达式分析采用分层函数处理优先级。`_parse_expression()` 进入比较表达式层，比较表达式调用加减表达式，加减表达式调用乘除表达式，乘除表达式再调用因子和基本表达式。这样可以自然保证乘除优先于加减，加减优先于比较。例如 `a + b * c` 会先在 `term` 层构造 `b * c`，再在 `additive` 层构造 `a + (b * c)`。

后缀表达式由 `_parse_postfix()` 处理，用来支持函数调用、数组下标和元组字段访问。例如 `add(a,b)` 会在识别 `IdentifierExpr(add)` 后继续看到左括号，从而构造 `CallExpr`；`xs[0]` 会构造 `IndexExpr`；`pair.0` 会构造 `FieldExpr`。

接上例：

```
fn test(mut a:i32) -> i32 {  
    let mut b:i32 = a + 1 * 2;  
    return b;  
}
```

`_parse_expression()` 解析 `a + 1 * 2`。

由于 `_parse_term()` 先处理乘除，所以 `1 * 2` 会先构造 `BinaryExpr`，再和 `a` 构造加法 `BinaryExpr`。

这一步结束后得到 AST 形态：

```
LetStmt  
name: b  
initializer:  
  BinaryExpr +  
    left: IdentifierExpr(a)  
    right:  
      BinaryExpr *  
        left: NumberExpr(1)  
        right: NumberExpr(2)
```

五、语义分析设计

语义分析模块对应 `src/semantic.py`。

语法分析不能处理的句子结构外的错误，语义分析来判断。

例：`a = 1;` 是语法上的合法语句，但是如果 `a` 没有声明，语义上就是错误的。

5.1 语义分析器设计目标

5.1.1 函数符号表检查

遍历所有函数定义，收集函数名、参数类型、返回类型，检查函数是否重复定义。

5.1.2 变量作用域管理

使用作用域栈 `scopes` 管理变量。进入代码块时压入新作用域，离开代码块时弹出作用域。

检查变量是否重复定义、是否未定义就使用。

5.1.3 类型检查

检查变量初始化、赋值、`return`、函数调用参数、数组元素、元组字段、引用和解引用的类型是否匹配。

5.1.4 可变性检查

检查不可变变量是否被赋值，`&mut` 是否借用了可写左值。

5.1.5 控制流相关检查

检查 `break`、`continue` 是否出现在循环内部；检查 `loop` 表达式中 `break value` 的类型是否一致。

5.2 语义分析器输入输出

语义分析在 `AST` 的基础上进一步检查程序的上下文规则、类型规则和作用域规则。

5.3 语义分析器设计逻辑

语义分析入口为 `analyze(program)`。该函数首先调用 `_collect_functions()` 遍历 `Program` 中的所有 `FunctionDecl`，将函数名、参数类型和返回类型登记到函数符号表中。这样做的原因是函数调用可能出现在函数定义之前，先收集函数信息可以让后续函数调用检查不依赖源码顺序。

完成函数收集后，语义分析器逐个分析函数。`_analyze_function()` 会建立函数作用域，将形参加入当前作用域，并记录当前函数的返回类型。随后调用 `_analyze_block()` 分析函数体。代码块分析过程中会依次检查每条语句，并在需要时创建新的作用域。

5.4 语义分析器设计与关键函数

语句分析由 `_analyze_statement()` 完成。对于 `LetStmt`，语义分析器检查初始化表达式类型是否与显式声明类型一致，并将变量加入当前作用域；对于 `AssignStmt`，先通过 `_lvalue_info()` 判断左值是否可写，再检查左右两侧类型是否匹配；对于 `ReturnStmt`，检查返回表达式类型是否与当前函数声明的返回类型一致；对于 `IfStmt`、`WhileStmt` 和 `ForStmt`，检查条件表达式或迭代表达式是否符合要求，并递归分析内部代码块。

表达式类型由 `_type_of_expr()` 推导。数字表达式返回 `i32` 类型，标识符表达式需要从作用域栈中查找变量，二元表达式要求左右两侧都是 `i32`，函数调用表达式需要检查函数是否存在、实参数量是否匹配、实参类型是否与形参类型一致。数组、元组、引用、解引用、下标访问和字段访问也都在该函数中完成基本类型推导。

六、测试设计

6.1 测试设计设想

测试的目的是为了检测能否正确跑通流程，能否正确检测出错误的程序，能否见撤出错误的程序，输出结果是否符合预期。
测试由两部分构成：单文件运行和批量回归测试。

6.2 测试样例设计

分为四类：

6.2.1 应通过样例

放在 `examples/minimum/`、`examples/p1/`、`examples/p2/`、`examples/p3/` 中。
用于验证最低要求和扩展语法是否能正常通过。

6.2.2 应失败样例

包括 `err_lex.rs`、`err_parse.rs` 和 `examples/semantic_errors/`。
分别验证词法错误、语法错误和语义错误是否能被发现。

6.2.3 词法边界测试

直接在 `run_regression.py` 中构造源码片段，检查 `Token` 列表是否符合预期。
例如 `if123` 应识别为标识符，而 `if=123` 应识别为 `IF EQ NUMBER`。

6.2.4 AST 结构测试

不仅检查程序能否通过，还检查解析出的 `AST` 是否包含预期节点。
例如检查 `else if` 是否生成 `ElseIf` 节点，`for 1..n` 是否生成 `RangeExpr`，数组访问是否生成 `IndexExpr`。

七、实验结果

统计通过样例如下：

最低要求覆盖：19 条
0.1、0.2、0.3：3 条
1.1~1.5：5 条
2.0~2.2：3 条
3.1~3.5：5 条
4.1：1 条
5.0、5.1：2 条
扩展覆盖： 31 条
合计：50 条 通过：50 条

```
[PASS] pass examples\ok_minimal.rs
[PASS] pass examples\minimum\01_functions_and_returns.rs
[PASS] pass examples\minimum\02_variables_and_assignment.rs
[PASS] pass examples\minimum\03_expressions_and_calls.rs
[PASS] pass examples\minimum\04_if_and_while.rs
[PASS] pass examples\p1\01_let_initializers.rs
[PASS] pass examples\p1\02_else_if.rs
[PASS] pass examples\p1\03_loop_break_continue.rs
[PASS] pass examples\p1\04_for_range.rs
[PASS] pass examples\p2\01_references.rs
[PASS] pass examples\p2\02_arrays.rs
[PASS] pass examples\p2\03_tuples.rs
[PASS] pass examples\p3\01_expression_blocks.rs
[PASS] pass examples\p3\02_if_and_loop_expressions.rs
[PASS] fail examples\err_lex.rs
[PASS] fail examples\err_parse.rs
[PASS] fail examples\semantic_errors\duplicate_var.rs
[PASS] fail examples\semantic_errors\function_args.rs
[PASS] fail examples\semantic_errors\immutable_assignment.rs
[PASS] fail examples\semantic_errors\immutable_ref_write.rs
[PASS] fail examples\semantic_errors\return_type.rs
[PASS] fail examples\semantic_errors\type_mismatch.rs
[PASS] fail examples\semantic_errors\undefined_var.rs
[PASS] lex if123_is_ident
[PASS] lex if_eq_number
[PASS] lex new_keywords_and_dots
[PASS] ast p1_nodes
[PASS] ast p2_nodes
[PASS] ast p3_nodes
```

7.1 词法分析结果

要能正确区分关键字，标识符等。

7.1.1 关键字和标识符边界

识别 if123 与 if = 123

fn program_0_0() { if123;

```
if = 123;  
}
```

```
=== 词法分析结果 ===  
1:1    FN      fn  
1:4    IDENT   program_0_0  
1:15   LPAREN  (  
1:16   RPAREN  )  
1:18   LBRACE  {  
2:5    IDENT   if123  
2:10   SEMICOLON ;  
3:5    IF      if  
3:8    EQ      =  
3:10   NUMBER  123  
3:13   SEMICOLON ;  
4:1    RBRACE  }
```

```
fn program_2_1() {  
    let mut a;  
    let mut b:i32;  
    let c;  
    let d:i32;  
}
```

=== 词法分析结果 ===

1:1	FN	fn
1:4	IDENT	program_2_1
1:15	LPAREN	(
1:16	RPAREN	)
1:18	LBRACE	{
2:5	LET	let
2:9	MUT	mut
2:13	IDENT	a
2:14	SEMICOLON	;
3:5	LET	let
3:9	MUT	mut
3:13	IDENT	b
3:14	COLON	:
3:15	I32	i32
3:18	SEMICOLON	;
4:5	LET	let
4:9	IDENT	c
4:10	SEMICOLON	;
5:5	LET	let
5:9	IDENT	d
5:10	COLON	:
5:11	I32	i32
5:14	SEMICOLON	;
6:1	RBRACE	}
8:1	FN	fn

7.1.2 单字符符号和双字符符号

```
fn program_3_2() {  
    1 < 2;  
    3 > 4;  
    1 <= 2;  
    3 >= 2;  
    1 == 1;  
    1 != 2;  
}
```

```
15:1    FN      fn
15:4    IDENT   program_3_2
15:15   LPAREN  (
15:16   RPAREN  )
15:18   LBRACE  {
16:5    NUMBER  1
16:7    LT      <
16:9    NUMBER  2
16:10   SEMICOLON ;
17:5    NUMBER  3
17:7    GT      >
17:9    NUMBER  4
17:10   SEMICOLON ;
18:5    NUMBER  1
18:7    LTE     <=
18:10   NUMBER  2
18:11   SEMICOLON ;
19:5    NUMBER  3
19:7    GTE     >=
19:10   NUMBER  2
19:11   SEMICOLON ;
20:5    NUMBER  1
20:7    EQEQ    ==
20:10   NUMBER  1
20:11   SEMICOLON ;
21:5    NUMBER  1
21:7    NE      !=
21:10   NUMBER  2
21:11   SEMICOLON ;
22:1    RBRACE  }
24:1    FN      fn
```

7.1.3 箭头和减号边界

```
fn program_7_2(mut x:i32, mut y:i32) -> i32 {
    let mut t = x * x + x;
    t = t + x * y;
    t
}
```

=== 词法分析结果 ===

1:1	FN	fn
1:4	IDENT	program_7_2
1:15	LPAREN	(
1:16	MUT	mut
1:20	IDENT	x
1:21	COLON	:
1:22	I32	i32
1:25	COMMA	,
1:27	MUT	mut
1:31	IDENT	y
1:32	COLON	:
1:33	I32	i32
1:36	RPAREN	)
1:38	ARROW	->
1:41	I32	i32
1:45	LBRACE	{
2:5	LET	let
2:9	MUT	mut
2:13	IDENT	t
2:15	EQ	=
2:17	IDENT	x
2:19	STAR	*
2:21	IDENT	x
2:23	PLUS	+
2:25	IDENT	x
2:26	SEMICOLON	;
3:5	IDENT	t
3:7	EQ	=
3:9	IDENT	t
3:11	PLUS	+
3:13	IDENT	x
3:15	STAR	*
3:17	IDENT	y
3:18	SEMICOLON	;
4:5	IDENT	t
5:1	RBRACE	}
7:1	FN	fn

7.2 语法分析结果

7.2.1 基础结构展示

```
fn program_1_5() -> i32 {  
    return 1;  
}
```

Json 格式:

```
{
  "type": "FunctionDecl",
  "name": "program_1_5",
  "params": [],
  "return_type": {
    "type": "I32Type"
  },
  "body": {
    "type": "Block",
    "statements": [
      {
        "type": "ReturnStmt",
        "value": {
          "type": "NumberExpr",
          "value": 1
        }
      }
    ],
    "result": null
  }
}
```

AST 树形结构:

```
`-- FunctionDecl: program_1_5
   |-- Params: []
   |-- ReturnType: i32
   |-- Block
   |-- ReturnStmt
   |-- NumberExpr: 1
```

7.2.2 表达式优先级

```
fn program_7_2(mut x:i32, mut y:i32) -> i32 {
  let mut t = x * x + x;
  t = t + x * y;
  t
}
```

```
"type": "Program",
"declarations": [
  {
    "type": "FunctionDecl",
    "name": "program_7_2",
    "params": [
      {
        "type": "Param",
        "is_mut": true,
        "name": "x",
        "type_name": {
          "type": "I32Type"
        }
      },
      {
        "type": "Param",
        "is_mut": true,
        "name": "y",
        "type_name": {
          "type": "I32Type"
        }
      }
    ],
    "return_type": {
      "type": "I32Type"
    },
    "body": {
      "type": "Block",
      "statements": [
        {
          "type": "LetStmt",
          "is_mut": true,
          "name": "t",
          "type_name": null,
          "initializer": {
            "type": "BinaryExpr",
            "op": "+",
            "left": {
              "type": "BinaryExpr",
```

```

        "op": "*",
        "left": {
            "type": "IdentifierExpr",
            "name": "x"
        },
        "right": {
            "type": "IdentifierExpr",
            "name": "x"
        }
    },
    "right": {
        "type": "IdentifierExpr",
        "name": "x"
    }
}
},
{
    "type": "AssignStmt",
    "target": {
        "type": "IdentifierExpr",
        "name": "t"
    },
    "value": {
        "type": "BinaryExpr",
        "op": "+",
        "left": {
            "type": "IdentifierExpr",
            "name": "t"
        },
        "right": {
            "type": "BinaryExpr",
            "op": "*",
            "left": {
                "type": "IdentifierExpr",
                "name": "x"
            },
            "right": {
                "type": "IdentifierExpr",
                "name": "y"
            }
        }
    }
}
},
},
],
"result": {
    "type": "IdentifierExpr",
    "name": "t"
}
}
}

```

```

=== AST Tree ===
Program
|-- FunctionDecl: program_7_2
|   |-- Params: [mut x: i32, mut y: i32]
|   |-- ReturnType: i32
|   `-- Block
|       |-- LetStmt: mut t
|       |   |-- Type: None
|       |   `-- Initializer
|       |       |-- BinaryExpr: +
|       |       |   |-- BinaryExpr: *
|       |       |   |   |-- IdentifierExpr: x
|       |       |   |   `-- IdentifierExpr: x
|       |       |   `-- IdentifierExpr: x
|       |-- AssignStmt
|       |   |-- Target
|       |   |   `-- IdentifierExpr: t
|       |   `-- Value
|       |       |-- BinaryExpr: +
|       |       |   |-- IdentifierExpr: t
|       |       |   `-- BinaryExpr: *
|       |       |       |-- IdentifierExpr: x
|       |       |       `-- IdentifierExpr: y
|       `-- Result
|           `-- IdentifierExpr: t

```

7.2.3 控制结构

```

{
  "type": "Program",
  "declarations": [
    {
      "type": "FunctionDecl",
      "name": "program_4_2",
      "params": [
        {
          "type": "Param",
          "is_mut": false,
          "name": "a",
          "type_name": {
            "type": "I32Type"
          }
        }
      ],
      "return_type": {
        "type": "I32Type"
      },
      "body": {
        "type": "Block",
        "statements": [],

```

```
"result": {
  "type": "IfExpr",
  "condition": {
    "type": "BinaryExpr",
    "op": ">",
    "left": {
      "type": "IdentifierExpr",
      "name": "a"
    },
    "right": {
      "type": "NumberExpr",
      "value": 0
    }
  },
  "then_block": {
    "type": "Block",
    "statements": [
      {
        "type": "ReturnStmt",
        "value": {
          "type": "NumberExpr",
          "value": 1
        }
      }
    ],
    "result": null
  },
  "else_if_blocks": [],
  "else_block": {
    "type": "Block",
    "statements": [
      {
        "type": "ReturnStmt",
        "value": {
          "type": "NumberExpr",
          "value": 0
        }
      }
    ],
    "result": null
  }
}
}
```

```

=== AST Tree ===
Program
|-- FunctionDecl: program_4_2
|   |-- Params: [a: i32]
|   |-- ReturnType: i32
|   |-- Block
|       |-- Result
|           |-- IfExpr
|               |-- Condition
|                   |-- BinaryExpr: >
|                       |-- IdentifierExpr: a
|                       |-- NumberExpr: 0
|               |-- Then
|                   |-- Block
|                       |-- ReturnStmt
|                           |-- NumberExpr: 1
|               |-- Else
|                   |-- Block
|                       |-- ReturnStmt
|                           |-- NumberExpr: 0

```

7.2.4 错误

缺少分号:

```

fn main() {
    return 1
}

```

```

分析失败: 3:1 return 语句末尾缺少 ';', 当前符号: '}'
    }
    ^

```

括号或大括号不匹配:

```

fn main() {
    if 1 > 0 {
        return 1;
    }
}

```

```

分析失败: 4:2 语句块未闭合, 缺少 '}', 当前符号: 'EOF'
    }
    ^

```

7.3 语义分析

7.3.1 语义正确案例

```

fn program_7_1(mut x:i32, mut y:i32) {
    let mut z = {
        let mut t = x * x + x;
    }
}

```

```
t = t + x * y;
t
};
}
```

语义分析通过。
分析成功：词法、语法与语义均通过。

7.3.2 变量重复定义

```
fn bad_duplicate() {
    let a = 1;
    let a = 2;
}
```

```
=== AST Tree ===
Program
  -- FunctionDecl: bad_duplicate
    -- Params: []
    -- ReturnType: None
    -- Block
      -- LetStmt: a
        -- Type: None
        -- Initializer
          -- NumberExpr: 1
      -- LetStmt: a
        -- Type: None
        -- Initializer
          -- NumberExpr: 2
```

分析失败：变量重复定义：a

7.3.3 参数匹配

```
fn add(mut a:i32, mut b:i32) -> i32 {
    return a + b;
}

fn bad_args() {
    add(1);
}
```

```

=== AST Tree ===
Program
|-- FunctionDecl: add
|   |-- Params: [mut a: i32, mut b: i32]
|   |-- ReturnType: i32
|   |-- Block
|       |-- ReturnStmt
|           |-- BinaryExpr: +
|               |-- IdentifierExpr: a
|               |-- IdentifierExpr: b
|-- FunctionDecl: bad_args
|   |-- Params: []
|   |-- ReturnType: None
|   |-- Block
|       |-- ExprStmt
|           |-- CallExpr: add
|               |-- NumberExpr: 1

```

分析失败：函数 add 实参数量不匹配

7.3.4 赋值规则

```

fn bad_immutable() {
    let a = 1;
    a = 2;
}

```

```

=== AST Tree ===
Program
|-- FunctionDecl: bad_immutable
|   |-- Params: []
|   |-- ReturnType: None
|   |-- Block
|       |-- LetStmt: a
|           |-- Type: None
|           |-- Initializer
|               |-- NumberExpr: 1
|       |-- AssignStmt
|           |-- Target
|               |-- IdentifierExpr: a
|           |-- Value
|               |-- NumberExpr: 2

```

分析失败：不能给不可变左值赋值

7.3.5 函数缺少返回表达式

```

fn bad_return() -> i32 {
    return;
}

```

```
=== AST Tree ===
```

```
Program
```

```
`-- FunctionDecl: bad_return
```

```
  |-- Params: []
```

```
  |-- ReturnType: i32
```

```
  |-- Block
```

```
    |-- ReturnStmt
```

分析失败：当前函数声明了返回类型，return 语句缺少返回表达式

7.3.6 赋值语句左右不匹配

```
fn bad_type_mismatch() {  
    let mut a:i32 = 1;  
    a = [1,2,3];  
}
```

```
=== AST Tree ===
```

```
Program
```

```
`-- FunctionDecl: bad_type_mismatch
```

```
  |-- Params: []
```

```
  |-- ReturnType: None
```

```
  |-- Block
```

```
    |-- LetStmt: mut a
```

```
      |-- Type: i32
```

```
      |-- Initializer
```

```
        |-- NumberExpr: 1
```

```
    |-- AssignStmt
```

```
      |-- Target
```

```
        |-- IdentifierExpr: a
```

```
      |-- Value
```

```
        |-- ArrayExpr
```

```
          |-- NumberExpr: 1
```

```
          |-- NumberExpr: 2
```

```
          |-- NumberExpr: 3
```

分析失败：赋值语句左右类型不匹配：期望 i32，实际 [i32;3]

7.3.7 未定义错误

```
fn bad_undefined() {  
    a;  
}
```

```
=== AST Tree ===  
Program  
  |-- FunctionDecl: bad_undefined  
    |-- Params: []  
    |-- ReturnType: None  
    |-- Block  
      |-- ExprStmt  
        |-- IdentifierExpr: a
```

分析失败：使用未定义变量：a

八、总结

本次大作业让我们对编译前端有了更具体的认识。最开始阅读类 Rust 词法和语法规则时，我们更多停留在“看懂产生式”的层面；真正实现之后，才逐渐理解词法分析为什么要先把字符流转换成 Token，语法分析为什么要在 Token 基础上进一步构造 AST，以及语义分析为什么需要结合上下文判断变量、类型和作用域等问题。项目过程中，我们也遇到了关键字与标识符边界、表达式左递归改写、运算符优先级处理、语法错误定位和语义错误检查等问题。解决这些问题的过程，让课堂上的 LL(1)、递归下降、符号表等概念变得更加清晰。总体来说，这次实验让我们收获很多，也让我们意识到编译原理需要在理论和实践中反复理解。接下来我们会在这门课的学习中继续努力，也会在今后的专业学习中保持认真探索和持续改进的态度。